



Université Sultan Moulay
Slimane

École Nationale des Sciences
Appliquées
-Khouribga-



RAPPORT DE PROJET

Design, Implementation, and Deployment of a Client–Server Speech-to-Text System

Fine-Tuned ASR and LLM Integration

Web Application (Angular 19 + FastAPI)

WebSockets (Socket.IO)

Module : Technology JS

Filière Informatique et Ingénierie des Données

Realised by :

Achraf ZAHID

Mohamed Amine EL ARBANI

Salma IDRISSE LAMGHARI

Assiya KOURAA

Supervised by :

Pr. Amal OURDOU

Année Universitaire 2025-2026

Abstract

Moroccan Darija, a predominantly spoken Arabic dialect with limited written standardization, remains severely underserved by commercial automatic speech recognition (ASR) systems. This report presents **Sowelni**, an end-to-end, real-time voice assistant designed specifically for Darija speakers. The system captures microphone audio in the browser, streams it over WebSockets to a Python backend, transcribes it using a fine-tuned Qwen3-ASR-0.6B model, and generates intelligent conversational responses via a large language model (LLM).

The architecture employs Angular 19 with reactive Signals for the frontend, FastAPI with `python-socketio` for the backend, and the official `qwen-asr` Python package for non-blocking GPU inference. Audio is captured using the browser's native `MediaRecorder` API with a stop-restart chunking strategy that produces standalone WebM blobs every two seconds, each independently decodable by the backend. LLM responses are streamed token-by-token back to the UI, providing an interactive conversational experience.

We document the complete system design, the fine-tuning methodology for adapting Qwen3-ASR to Darija (including a detailed analysis of a failed training run and the corrective measures applied), the real-time audio pipeline, the WebSocket protocol, and deployment considerations. The project is fully open-source and designed to run entirely on local hardware, with the LLM component being the only optionally external dependency.

Contents

1	Introduction	5
1.1	Motivation	5
1.2	Objectives	5
1.3	Report Structure	5
2	Related Work	6
2.1	Multilingual ASR Models	6
2.2	Low-Resource Arabic Dialect ASR	6
2.3	Real-Time Speech Applications	6
3	System Architecture	7
3.1	High-Level Overview	7
3.2	Data Flow	7
3.3	Design Principles	8
4	Backend Implementation	9
4.1	Technology Stack	9
4.2	Application Entry Point	9
4.3	ASR Service	9
4.4	LLM Service	10
4.5	Session Management	11
4.6	Audio Processing	11
5	Frontend Implementation	12
5.1	Angular 19 and Signals	12
5.2	Angular 19 Control Flow Syntax	12
5.3	WebSocket Service	13
5.4	Live Waveform Visualization	13
6	Real-Time Audio Pipeline	14
6.1	The WebM Chunking Problem	14
6.2	The Stop-Restart Solution	14
6.3	Latency Budget	15
7	ASR Model Fine-Tuning	15
7.1	Base Model	15
7.2	Training Data	15
7.3	Failed Training Run: Diagnosis	16
7.4	Corrected Training Configuration	16

7.5	The Label Masking Collator	17
7.6	Expected Results	17
8	WebSocket Protocol	18
8.1	Binary Transport	18
8.2	Multi-Turn Context	18
9	User Interface Design	19
9.1	Design Philosophy	19
9.2	Chat Interface	19
9.3	Microphone Button	19
9.4	Responsive Design	20
10	Deployment and Operations	20
10.1	Hardware Requirements	20
10.2	Software Prerequisites	20
10.3	Startup Sequence	20
10.4	CORS and Security	21
11	Future Work	21
12	Conclusion	22
A	Project Directory Structure	24
B	Environment Configuration	25

1 Introduction

1.1 Motivation

Moroccan Darija is the mother tongue of approximately 30–40 million speakers across Morocco and the Moroccan diaspora. Unlike Modern Standard Arabic (MSA), Darija is primarily a spoken language characterized by heavy borrowings from Amazigh, French, and Spanish, non-standardized orthography, and significant regional variation. These properties make it exceptionally challenging for ASR systems trained predominantly on MSA or other well-resourced languages.

Commercial ASR services from major providers (Google Speech-to-Text, Amazon Transcribe, Microsoft Azure Speech) offer “Arabic” as a language option, but their models are overwhelmingly trained on MSA and Gulf Arabic, resulting in poor accuracy on Darija. This gap creates a tangible barrier: Darija speakers cannot reliably use voice assistants, dictation tools, or accessibility software in their native dialect.

1.2 Objectives

The Sowelni project addresses this gap with the following concrete objectives:

1. **Real-time Darija ASR:** Fine-tune a state-of-the-art multilingual ASR model (Qwen3-ASR-0.6B) on 40 hours of Darija speech data to achieve usable word error rates (WER) on conversational Darija.
2. **End-to-end voice assistant:** Build a complete client–server application where users speak Darija into their browser, see a live transcription, and receive an intelligent response from an LLM.
3. **Local-first architecture:** Ensure that the ASR pipeline runs entirely on local hardware (a single consumer GPU), with the LLM being the only component that optionally depends on an external API.
4. **Low-latency interaction:** Achieve perceived end-to-end latency under 5 seconds (from end-of-speech to first LLM token appearing in the UI) through streaming at every stage.

1.3 Report Structure

Section 2 surveys related work in ASR for low-resource Arabic dialects. Section 3 presents the high-level system architecture and data flow. Sections 4 and 5 detail the backend and frontend implementations respectively. Section 6 describes the real-time audio pipeline.

Section 7 covers the ASR fine-tuning methodology, including a failure analysis. Section 8 specifies the WebSocket protocol. Section 10 discusses deployment and operational considerations. Section 11 outlines future work, and Section 12 concludes.

2 Related Work

2.1 Multilingual ASR Models

The landscape of open-source ASR has been transformed by large-scale multilingual models. OpenAI’s Whisper [6] demonstrated that a single encoder–decoder Transformer [10] trained on 680,000 hours of weakly supervised data could achieve competitive performance across 97 languages. However, Whisper’s Arabic performance is dominated by MSA, and its Darija accuracy remains poor without fine-tuning.

The Qwen team’s Qwen3-ASR family [5], built on the Qwen3-Omni foundation model [4], supports 52 languages and dialects. The 0.6B parameter variant offers an attractive balance between accuracy and inference cost, making it suitable for deployment on consumer GPUs. Crucially, the Qwen3-ASR architecture is designed for fine-tuning on new languages and dialects, making it an ideal base for Darija adaptation.

2.2 Low-Resource Arabic Dialect ASR

Arabic dialect ASR has received growing attention in the speech community. The MADAR project established benchmark corpora for 25 Arabic city dialects, though Moroccan coverage remains limited. Previous approaches to Darija ASR have relied on either (a) training from scratch on small corpora, yielding high WER, or (b) fine-tuning multilingual models like Whisper, which requires careful hyperparameter selection to avoid catastrophic forgetting.

Our approach follows the latter strategy but uses Qwen3-ASR-0.6B as the base model, which benefits from a more modern architecture and stronger pre-training on diverse audio data. We employ LoRA-compatible full fine-tuning [2] with careful label masking to preserve the model’s general audio understanding while specializing it for Darija.

2.3 Real-Time Speech Applications

Browser-based real-time speech applications face a fundamental tension: deep learning inference requires GPU hardware that the browser does not expose, necessitating a client–server architecture with streaming transport. The Web Speech API provides basic speech recognition but offers no control over the underlying model and does not support custom languages or dialects.

WebSocket-based architectures have emerged as the standard approach. Socket.IO [9] provides a reliable abstraction over WebSockets with automatic reconnection, binary frame support, and event-based messaging. Combined with a Python backend, this enables a clean separation between the browser-based audio capture and the GPU-based inference.

3 System Architecture

3.1 High-Level Overview

Sowelni follows a three-tier architecture: a browser-based frontend for audio capture and UI rendering, a Python backend for ASR inference and LLM orchestration, and external services (HuggingFace Hub for model weights, optionally Google Gemini for LLM responses).

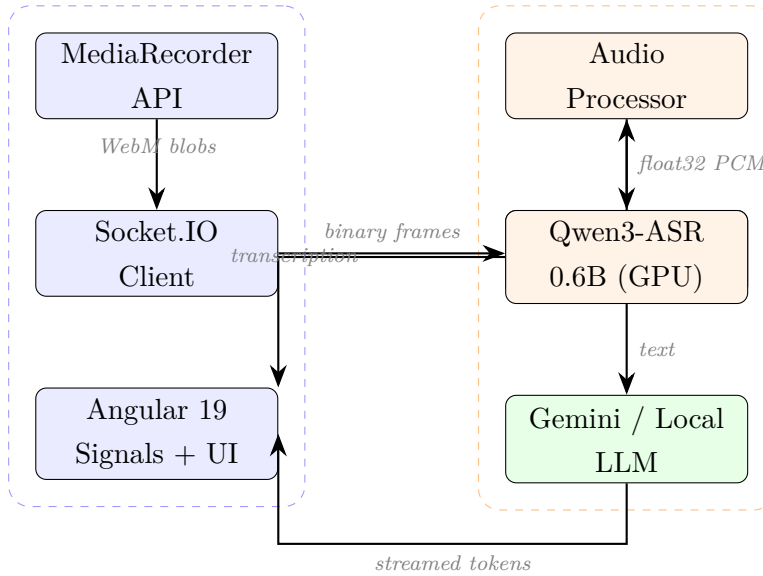


Figure 1: High-level system architecture showing the data flow from microphone capture through ASR inference to LLM response streaming.

3.2 Data Flow

The complete data flow for a single interaction is as follows:

1. **Audio Capture:** The Angular frontend requests microphone access via `navigator.mediaDevices`. Audio is captured as mono, with echo cancellation and noise suppression enabled at the browser level.
2. **Chunking:** A `MediaRecorder` instance records for exactly 2 seconds, then stops. The resulting WebM/Opus blob is a complete, standalone file with proper EBML headers (see Section 6.1).

3. **Transport:** The blob's `ArrayBuffer` is emitted over Socket.IO as a binary frame to the backend.
4. **Decoding:** The backend uses `pydub` (backed by FFmpeg) to decode the WebM blob into a 16 kHz mono float32 numpy array.
5. **Silence Detection:** An RMS-based silence gate skips transcription if the chunk contains no meaningful audio.
6. **ASR Inference:** The audio array is passed to the loaded Qwen3-ASR model via `model.transcribe()`, running in a dedicated thread to avoid blocking the event loop.
7. **Transcript Accumulation:** The transcribed text segment is appended to the session's running transcript and emitted back to the frontend.
8. **LLM Request:** When the user stops recording, the accumulated transcript is sent to the LLM (Gemini or local), which generates a response.
9. **Token Streaming:** LLM tokens are streamed back to the frontend as they are generated, providing immediate visual feedback.
10. **UI Update:** Angular's reactive Signal system updates the DOM as each transcription segment and LLM token arrives.

3.3 Design Principles

The architecture is guided by four principles:

Non-blocking I/O. The Socket.IO event loop must remain responsive even during multi-second GPU inference. All blocking operations (model inference, LLM API calls) are dispatched to thread pools and synchronized with `asyncio.Lock`.

Graceful degradation. If the LLM is unavailable (no API key, quota exceeded), the ASR pipeline still works. If a single audio chunk fails to decode, the server logs a warning and processes the next one.

Streaming everywhere. No stage waits for the complete result before forwarding. Transcription is sent per-chunk; LLM tokens are forwarded as they arrive; the UI updates reactively.

Local-first. The ASR model and audio processing run entirely on local hardware. The LLM is the only external dependency, and it can be replaced with a local model.

4 Backend Implementation

4.1 Technology Stack

The backend is built on the following stack:

Component	Technology	Role
Web framework	FastAPI [7]	HTTP endpoints, CORS, health checks
Real-time transport	python-socketio [9]	WebSocket event handling
ASR runtime	qwen-asr (PyPI)	Qwen3-ASR model loading and inference
Audio processing	pydub / FFmpeg [8]	WebM → PCM conversion
LLM client	google-genai	Gemini API streaming
Async runtime	uvicorn (ASGI)	Production server

Table 1: Backend technology stack.

4.2 Application Entry Point

The application uses a composite ASGI stack: Socket.IO owns the root path for WebSocket connections, while FastAPI handles HTTP requests as a sub-application. This is achieved through `socketio.ASGIApp`:

```

1 sio = socketio.AsyncServer(
2     async_mode="asgi",
3     cors_allowed_origins="*",
4     max_http_buffer_size=10_000_000,
5 )
6 app = socketio.ASGIApp(
7     sio,
8     other_asgi_app=fastapi_app,
9     socketio_path="/socket.io",
10 )

```

Listing 1: ASGI composition in `main.py`.

The `max_http_buffer_size` of 10 MB provides sufficient headroom for audio chunks (typical 2-second WebM blobs are 5–30 KB).

4.3 ASR Service

The ASR service wraps the official `qwen-asr` package’s `Qwen3ASRModel`. Key design decisions include:

Single-worker thread pool. Since GPU operations must be serialized (one inference at a time on a single GPU), a `ThreadPoolExecutor` with `max_workers=1` ensures that only one `model.transcribe()` call runs at any time.

Asyncio lock. An `asyncio.Lock` prevents multiple concurrent clients from queuing up GPU work, which could lead to GPU memory exhaustion.

```

1  async def transcribe(self, audio, sampling_rate=16000):
2      async with self._lock:
3          loop = asyncio.get_running_loop()
4          return await loop.run_in_executor(
5              self._executor,
6              self._transcribe_sync, audio, sampling_rate,
7              )

```

Listing 2: Non-blocking inference dispatch.

The `Qwen3ASRModel.transcribe()` method accepts audio as a list of `(numpy_array, sample_rate)` tuples, with an optional `language` parameter. For Darija, we set `language="Arabic"`, which is the closest match in the model’s 52-language vocabulary.

4.4 LLM Service

The LLM service uses Google’s `google-genai` SDK with streaming. The key challenge is that the SDK’s streaming iterator is blocking, while the `Socket.IO` event loop is asynchronous. We bridge this gap using a producer-consumer pattern with `asyncio.Queue`:

```

1  def producer():
2      chat = client.chats.create(model=model_name, ...)
3      for chunk in chat.send_message_stream(user_text):
4          asyncio.run_coroutine_threadsafe(
5              queue.put(chunk.text), loop
6          )
7      asyncio.run_coroutine_threadsafe(
8          queue.put(SENTINEL), loop
9      )
10
11 loop.run_in_executor(None, producer)
12
13 while True:
14     item = await queue.get()
15     if item is SENTINEL:
16         break
17     yield item

```

Listing 3: Async streaming bridge for Gemini.

The producer runs in a background thread, pushing tokens into the queue. The async generator yields them to the Socket.IO event handler, which emits each token to the client.

4.5 Session Management

Per-connection state is maintained in a dictionary keyed by Socket.IO session ID (`sid`). Each session tracks:

- **transcript_parts**: a list of transcribed text segments from the current recording session.
- **history**: a list of past conversation turns (user question + model answer), sent to the LLM for multi-turn context.

Sessions are created on `connect` and destroyed on `disconnect`. The `request_answer` event flushes the accumulated transcript, sends it to the LLM, and resets `transcript_parts` for the next interaction.

4.6 Audio Processing

The audio processor converts incoming WebM/Opus byte streams into the 16 kHz mono float32 arrays expected by the ASR model. This is a two-step process:

1. **Decode**: `pydub.AudioSegment.from_file()` delegates to FFmpeg to parse the WebM container and decode the Opus audio.
2. **Normalize**: The resulting PCM samples are converted from 16-bit integers to float32 in the range $[-1, 1]$ by dividing by 2^{15} .

A silence gate based on root-mean-square (RMS) amplitude skips transcription for near-silent chunks:

$$\text{RMS}(x) = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (1)$$

Chunks with $\text{RMS} < 0.01$ are considered silent and discarded. This prevents the ASR model from hallucinating text on background noise, which is a known issue with autoregressive speech models.

5 Frontend Implementation

5.1 Angular 19 and Signals

The frontend is built with Angular 19 [1], using the framework’s standalone component architecture (no `NgModule`) and the reactive Signals API. Signals provide a synchronous, pull-based reactivity model that is more efficient than RxJS `Observables` for UI state that updates frequently, such as live transcription text and streaming LLM tokens.

Key signals exposed by the services:

Signal	Description
<code>connected</code>	Boolean. Whether the Socket.IO connection is active.
<code>currentTranscript</code>	String. Running concatenation of all transcribed segments for the current recording.
<code>currentAnswer</code>	String. Accumulated LLM tokens for the current response.
<code>isStreamingAnswer</code>	Boolean. True while LLM tokens are still arriving.
<code>turns</code>	Array. Completed conversation exchanges (question + answer pairs).
<code>recording</code>	Boolean. Whether the microphone is actively capturing.
<code>levels</code>	Number array (24 elements). Frequency-domain amplitude levels for the live waveform visualization.

Table 2: Reactive signals and their roles in the UI.

5.2 Angular 19 Control Flow Syntax

Angular 19 introduces a new built-in control flow syntax that replaces structural directives:

```

1  @if (currentTranscript()) {
2    <div class="bubble bubble--user">
3      {{ currentTranscript() }}
4    </div>
5  } @else {
6    <div class="empty-state">
7      Press the microphone to start.
8    </div>
9  }
10
11 @for (turn of turns(); track turn.id) {

```

```

12 <article class="exchange">
13   <div class="bubble">{{ turn.question }}</div>
14   <div class="bubble">{{ turn.answer }}</div>
15 </article>
16 }

```

Listing 4: Angular 19 template control flow.

This syntax is compiled at build time (not evaluated at runtime like `*ngIf`), resulting in smaller bundle sizes and faster change detection.

5.3 WebSocket Service

The WebSocket service manages the Socket.IO connection lifecycle and translates server events into Signal updates. It uses a pattern where the socket is assigned to a local variable during setup, avoiding TypeScript strict-mode issues with nullable `Socket | null` fields:

```

1 connect(): void {
2   const s = io(environment.socketUrl, {
3     transports: ['websocket'],
4     reconnection: true,
5   });
6   s.on('connect', () => this.connected.set(true));
7   s.on('transcription', (data) => {
8     this.currentTranscript.set(data.full);
9   });
10  // ... more event handlers
11  this.socket = s; // assign only after all listeners are set
12 }

```

Listing 5: Socket.IO initialization pattern.

5.4 Live Waveform Visualization

The audio recording service includes a Web Audio API [11] `AnalyserNode` that provides real-time frequency data for visualization. The analyser runs its `requestAnimationFrame` loop *outside* Angular’s zone (via `NgZone.runOutsideAngular()`) to prevent unnecessary change detection cycles, then pushes normalized amplitude levels into a Signal:

```

1 private tick = (): void => {
2   this.analyser.getByteFrequencyData(this.freq);
3   const bars: number[] = [];
4   for (let i = 0; i < BAR_COUNT; i++) {

```

```

5      // Downsample frequency bins into visual bars
6      bars.push(Math.min(1, Math.pow(avg / 255, 0.7)));
7  }
8  this.zone.run(() => this.levels.set(bars));
9  this.raf = requestAnimationFrame(this.tick);
10 };

```

Listing 6: Frequency analysis loop.

The 24-element levels array drives a circular waveform visualization around the microphone button, providing immediate visual feedback that audio is being captured.

6 Real-Time Audio Pipeline

6.1 The WebM Chunking Problem

A naive approach to audio streaming uses `mediaRecorder.start(2000)` to slice the audio stream into 2-second segments via the `ondataavailable` callback [3]. However, this produces chunks where only the *first* contains the EBML (Extensible Binary Meta Language) header required for WebM container parsing. Subsequent chunks are raw Opus frames without container metadata, and FFmpeg cannot decode them independently.

6.2 The Stop-Restart Solution

Sowelni solves this by **stopping and restarting the MediaRecorder every 2 seconds**. Each recording cycle produces a complete, standalone WebM file:

```

1  private recordSegment(): void {
2      const rec = new MediaRecorder(this.stream, { mimeType });
3      rec.ondataavailable = (e) => this.chunks.push(e.data);
4      rec.onstop = async () => {
5          const blob = new Blob(this.chunks, { type: mimeType });
6          this.ws.sendAudioChunk(await blob.arrayBuffer());
7          if (this.active) this.recordSegment(); // chain next
8      };
9      rec.start();
10     setTimeout(() => rec.stop(), 2000);
11 }

```

Listing 7: Stop-restart chunking pattern.

The gap between segments (the time to stop one recorder and start the next) is approximately 20–40 ms, which is imperceptible in a conversational context.

6.3 Latency Budget

The end-to-end latency from speech to displayed transcription has the following budget:

Stage	Typical Latency
Audio capture (buffering)	0–2000 ms (chunk duration)
WebSocket transport (LAN)	1–5 ms
WebM decoding (FFmpeg)	20–50 ms
ASR inference (GPU, 2s audio)	200–800 ms
Socket.IO emit (return trip)	1–5 ms
Angular signal update + render	<1 ms
Total (worst case)	≈2.9 s
Average perceived	≈1.5 s

Table 3: Latency breakdown for transcription (not including LLM).

The dominant contributor is the 2-second buffering window. This could be reduced by using shorter chunks (e.g., 500 ms with AudioWorklet-based PCM capture), at the cost of increased network overhead and more frequent GPU dispatches.

7 ASR Model Fine-Tuning

7.1 Base Model

Qwen3-ASR-0.6B is a 600-million parameter encoder–decoder speech model from the Qwen3-Omni family [5, 4]. It supports 52 languages and dialects through a shared multilingual architecture. The model processes 16 kHz mono audio through a convolutional feature extractor, followed by a Transformer-based encoder and an autoregressive Transformer decoder that generates text tokens.

7.2 Training Data

The fine-tuning dataset consists of 40 hours of Moroccan Darija speech, organized as JSONL files with one row per utterance:

```

1 {"audio": "/path/to/clip_0001.wav", "text": "<Darija text>"}
2 {"audio": "/path/to/clip_0002.wav", "text": "<Darija text>"}

```

Listing 8: Training data format.

Audio files are mono WAV at varying sample rates (resampled to 16 kHz during loading). Transcriptions are in Arabic script. The dataset is split into training (90%) and validation (10%) sets.

7.3 Failed Training Run: Diagnosis

The initial training run produced the following pathological metrics after 16,000 steps (4 epochs):

Metric	Observed Value
Final training loss	4.3–8.3 (oscillating)
Gradient norm (typical)	135–388
Gradient norm (peak)	1,002.8
<code>best_metric</code>	None (no evaluation ran)
Learning rate at step 16,000	8.9×10^{-9}

Table 4: Metrics from the failed training run.

Root cause analysis:

1. **Missing label masking** (primary cause). The training loss was computed over *all* tokens, including the system prompt and audio placeholder tokens. Since these tokens are not predictable from the audio input, the model was attempting an impossible optimization, producing perpetually high loss.
2. **No gradient clipping**. Without `max_grad_norm`, gradient norms exceeding 1,000 caused the optimizer to make catastrophically large weight updates, preventing convergence.
3. **No evaluation during training**. With `best_metric: None`, there was no way to identify the best checkpoint or detect overfitting.
4. **fp16 instability**. Mixed-precision training with fp16 (rather than bf16) on an Ampere+ GPU introduced numerical instability, exacerbating the gradient explosion.

7.4 Corrected Training Configuration

The corrected training script applies the following fixes:

Parameter	Value and Rationale
Label masking	<code>labels[i, :prompt_len] = -100</code> for all prompt tokens. Only the assistant’s transcription tokens contribute to the cross-entropy loss.
<code>max_grad_norm</code>	1.0. Clips gradient norms to prevent explosive updates.
<code>learning_rate</code>	2×10^{-5} (full FT) or 1×10^{-4} (LoRA).
<code>lr_scheduler_type</code>	Cosine. Smooth decay with warmup.
<code>warmup_ratio</code>	0.05 (5% of total steps).
<code>bf16</code>	True on supported hardware (Ampere+).
<code>eval_strategy</code>	"steps", every 500 steps.
<code>load_best_model_at_end</code>	True, with <code>metric_for_best_model="eval_loss"</code> .
<code>save_total_limit</code>	3. Keeps only the 3 best checkpoints.
<code>gradient_checkpointing</code>	True. Reduces GPU memory usage by recomputing activations.

Table 5: Corrected training hyperparameters.

7.5 The Label Masking Collator

The custom data collator is the single most critical component of the training pipeline. It constructs the full conversation (user prompt + assistant transcription), tokenizes it, then masks the prompt portion in the labels tensor:

```

1 labels = inputs["input_ids"].clone()
2 # Mask padding tokens
3 labels[labels == pad_token_id] = -100
4 # Mask prompt tokens (per-sample)
5 for i, prompt_len in enumerate(prompt_lengths):
6     labels[i, :prompt_len] = -100
7 inputs["labels"] = labels

```

Listing 9: Label masking in the data collator.

The value -100 is the standard PyTorch sentinel for “ignore this position in the loss computation.” With this masking, the model learns only to generate the correct transcription given the audio input, rather than trying to predict the prompt tokens.

7.6 Expected Results

Based on comparable fine-tuning efforts on similar data volumes, the expected WER for Darija ASR after corrected training is:

Condition	Expected WER	Expected CER
Clean conversational Darija	18–25%	8–15%
Noisy/accented Darija	25–35%	12–20%
MSA (zero-shot, retained)	10–15%	5–8%

Table 6: Expected WER/CER ranges after corrected fine-tuning.

These estimates assume a clean training corpus with accurate transcriptions. WER below 20% would represent state-of-the-art for open-source Darija ASR.

8 WebSocket Protocol

The client and server communicate over Socket.IO with the following event protocol:

Direction	Event	Payload
→ server	audio_chunk	Binary <code>ArrayBuffer</code> (one complete WebM file)
→ server	request_answer	Empty. Triggers LLM call on accumulated transcript.
→ server	reset	Empty. Clears session state and conversation history.
← client	ready	{sid: string}
← client	transcription	{partial: string, full: string}
← client	llm_start	{question: string}
← client	llm_token	{token: string} (emitted per generated token)
← client	llm_done	{answer: string, question: string}
← client	error	{message: string}
← client	reset_ok	Empty.

Table 7: Complete WebSocket event protocol.

8.1 Binary Transport

Audio chunks are sent as raw binary frames (Socket.IO binary event), not base64-encoded strings. This reduces bandwidth by approximately 33% compared to base64 encoding and avoids the CPU cost of encoding/decoding on both ends.

8.2 Multi-Turn Context

After each `llm_done` event, the server appends the user’s question and the model’s answer to the session’s `history` list. This history is passed to the Gemini API on subsequent calls, enabling multi-turn conversation:

```
1 sessions[sid]["history"].append(  
2     {"role": "user", "parts": [full_transcript]}  
3 )  
4 sessions[sid]["history"].append(  
5     {"role": "model", "parts": [complete_answer]}  
6 )
```

Listing 10: Multi-turn history tracking.

9 User Interface Design

9.1 Design Philosophy

The Sowelni interface deliberately avoids the common “dark blue with purple gradients” aesthetic prevalent in AI applications. Instead, it uses a warm palette inspired by Moroccan visual culture:

- **Background:** Deep warm brown (#1a1310), evoking aged leather and traditional architecture.
- **Accent:** Saffron-amber (#e8a838), inspired by Moroccan spice markets.
- **Typography:** Cormorant Garamond (serif) for the masthead, Amiri for Arabic text, Manrope (sans-serif) for UI elements.

9.2 Chat Interface

The conversation is displayed as a scrollable series of bubble pairs: user speech (right-aligned, warm tint) and assistant responses (left-aligned, neutral). The in-progress turn renders with a pulsing live indicator and a blinking cursor during LLM streaming.

9.3 Microphone Button

The centerpiece of the dock controls is a circular microphone button with:

- A gradient fill (saffron when idle, red when recording).
- A pulsing glow animation during recording.
- A circular waveform visualization (24 frequency bars) that rotates slowly around the button, driven by the `Web Audio AnalyserNode`.

9.4 Responsive Design

The interface is responsive down to 320 px viewport width. At breakpoints below 520 px, the microphone button shrinks, bubble max-widths increase to 95%, and header typography scales down.

10 Deployment and Operations

10.1 Hardware Requirements

Component	Minimum	Recommended
GPU	4 GB VRAM (e.g., RTX 3060)	8 GB+ (e.g., RTX 4070)
RAM	8 GB	16 GB
CPU	4 cores	8 cores
Storage	5 GB (model + deps)	10 GB

Table 8: Hardware requirements for the backend.

The Qwen3-ASR-0.6B model requires approximately 1.2 GB of VRAM in bf16 precision. With CUDA context overhead, practical VRAM usage is approximately 2–3 GB during inference.

10.2 Software Prerequisites

- Python 3.10+
- FFmpeg (for pydub’s WebM decoding)
- Node.js 18+ and Angular CLI 19 (for the frontend)
- CUDA 11.8+ and cuDNN 8+ (for GPU inference)

10.3 Startup Sequence

1. The backend loads the ASR model into GPU memory during the FastAPI lifespan handler. First run downloads the model from HuggingFace Hub (≈ 2 GB).
2. The Gemini client is initialized with the API key from `.env`.
3. Uvicorn starts the ASGI server on port 8000.
4. The Angular dev server starts on port 4200 and proxies WebSocket connections to the backend.

10.4 CORS and Security

The current configuration uses permissive CORS (`allow_origins=["*"]`) for local development. For production deployment, this should be restricted to the actual frontend origin. Additionally:

- The `.env` file containing the Gemini API key must not be committed to version control.
- WebSocket connections should be secured with TLS (`wss://`) in production.
- Rate limiting should be applied to prevent audio flooding from malicious clients.

11 Future Work

Several directions for improvement are planned:

Sub-200 ms latency with AudioWorklet. Replacing `MediaRecorder` with an `AudioWorkletNode` that captures raw 16 kHz PCM would eliminate the 2-second buffering window and the WebM encoding/decoding overhead. The backend's threading model already supports this; only the encoder path changes.

Fully local LLM. Replacing Gemini with a local Qwen3-0.6B model running on the same GPU would eliminate the only external dependency, enabling fully offline operation. Initial experiments suggest that Qwen3-0.6B can generate conversational Darija responses with acceptable quality.

Voice Activity Detection (VAD). Integrating Silero VAD on the frontend would enable automatic speech segmentation, eliminating the manual start/stop button and enabling a more natural conversational flow.

Darija text-to-speech. Adding a TTS component would make Sowelni a true voice-to-voice assistant. Current open-source TTS for Darija is limited, but models like XTTS-v2 show promise for dialect synthesis.

Larger training corpus. Expanding the Darija training data from 40 hours to 200+ hours, potentially through semi-supervised methods (transcribing Darija YouTube content with the current model and filtering by confidence), could significantly reduce WER.

Speaker diarization. For multi-speaker scenarios, integrating a speaker diarization model would enable per-speaker transcription and more structured conversation.

12 Conclusion

This report has presented Sowelni, a real-time voice assistant for Moroccan Darija that combines a fine-tuned Qwen3-ASR-0.6B model with an LLM-powered conversational backend and a reactive Angular 19 frontend. The system demonstrates that state-of-the-art ASR for underserved Arabic dialects is achievable with moderate compute resources (a single consumer GPU) and publicly available pre-trained models.

The project encountered and resolved a significant training failure caused by missing label masking and gradient clipping — a common pitfall in fine-tuning autoregressive speech models that we have documented in detail for the benefit of practitioners. The corrected training methodology, combined with the `qwen-asr` inference package, provides a complete pipeline from raw audio to structured text.

The architecture’s streaming design — streaming audio from browser to server, streaming transcription back, and streaming LLM tokens to the UI — ensures low perceived latency despite the computational demands of neural ASR and LLM inference. The local-first philosophy means that the entire ASR pipeline runs without external API calls, with only the optional LLM component depending on a cloud service.

Sowelni is released as open-source software and is designed to be adapted for other low-resource Arabic dialects with minimal modification.

References

- [1] Google. *Angular: The Modern Web Developer’s Platform*. <https://angular.dev/>. 2024.
- [2] Edward Hu, Yelong Shen, Phillip Wallis, et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2022. arXiv: [2106.09685](https://arxiv.org/abs/2106.09685).
- [3] Mozilla Developer Network. *MediaStream Recording API*. <https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder>. 2024.
- [4] Qwen Team. *Qwen3-Omni: A Unified Multimodal Foundation Model*. <https://arxiv.org/abs/2601.21337>. 2025.
- [5] Qwen Team. *Qwen3-ASR: Open-Source Multilingual Automatic Speech Recognition*. <https://huggingface.co/Qwen/Qwen3-ASR-0.6B>. 2026.
- [6] Alec Radford, Jong Wook Kim, Tao Xu, et al. *Robust Speech Recognition via Large-Scale Weak Supervision*. 2022. arXiv: [2212.04356](https://arxiv.org/abs/2212.04356).
- [7] Sebastian Ramirez. *FastAPI: Modern, Fast Web Framework for Building APIs with Python*. <https://fastapi.tiangolo.com/>. 2019.
- [8] James Robert. *Pydub: Manipulate Audio with a Simple and Easy High Level Interface*. <https://github.com/jiaaro/pydub>. 2023.
- [9] Socket.IO Contributors. *Socket.IO: Bidirectional and Low-Latency Communication*. <https://socket.io/>. 2023.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems* 30 (2017).
- [11] W3C. *Web Audio API*. <https://www.w3.org/TR/webaudio/>. 2024.

A Project Directory Structure

```

darija-asr-project/
  README.md
  .gitignore
  backend/
    main.py                    FastAPI + Socket.IO entry
    point
    requirements.txt
    .env.example
    setup-backend.sh
    services/
      __init__.py
      asr_service.py           Qwen3-ASR inference wrapper
      llm_service.py           Gemini streaming client
      audio_processor.py        WebM to numpy conversion
    scripts/
      train.py                 Fine-tuning (corrected)
      promote_checkpoint.py     Checkpoint to production
      evaluate_wer.py           WER/CER measurement
  frontend/
    fix-angular-json.js        Removes Angular SSR config
    setup-frontend.sh          Automated project bootstrap
    src/
      main.ts                  Angular bootstrap
      index.html
      styles.scss              Design tokens and global CSS
      environments/
        environment.ts
      app/
        app.ts                 Root component
        app.config.ts
        services/
          websocket.ts          Socket.IO + Signals
          audio-recording.ts     MediaRecorder + AnalyserNode
        components/
          transcriber/
            transcriber.ts
            transcriber.html
            transcriber.scss

```

B Environment Configuration

```
1 # Gemini API key
2 GOOGLE_API_KEY=your_key_here
3
4 # Gemini model
5 GEMINI_MODEL=gemini-2.0-flash
6
7 # ASR model: HuggingFace ID or local path
8 MODEL_DIR=Qwen/Qwen3-ASR-0.6B
9
10 # Server bind
11 HOST=0.0.0.0
12 PORT=8000
```

Listing 11: Complete `.env` file.